

COMP 2404 -- Tutorial #1

C++ global functions and Makefiles

1. Learning Outcomes

After this tutorial, you will be able to:

- write a basic C++ program using correctly designed global functions, distributed over multiple files
- update a Makefile to compile and link a program

NOTE: Tutorials must be completed individually; collaboration will be reported as an instructional offence

2. Required Background

To begin this tutorial, you must understand the following concepts from the course material:

- COMP 2401: basic Linux commands, Makefiles
- section 1.1: compiling, linking, and Makefiles in Linux, source files, object files
- section 1.2: global functions in C++

If you are not familiar with this material, you **must** review it on your own before you can start this tutorial.

3. Tutorial Instructions

THE INSTRUCTIONS MUST BE COMPLETED IN ORDER.

1. If you are using your own laptop, make sure that you have installed VirtualBox and the extension pack. You must download the course VM from the [SCS web site](#), and you must import the VM into VirtualBox. The instructions for this process can be found [here](#).
2. Start up the VM for the course and log in. Create a `COMP2404` directory in your home directory, and a `Tutorials` sub-directory. If you need to refresh your Linux skills, you should consult your COMP 2401 tutorials. This page has a decent list of basic commands: [basic Linux commands](#)
3. From the tutorial page in *Brightspace*, download the tutorial archive file `T01.tar` into your `Tutorials` directory. Un-tar the file, using the `extract`, `verbose`, from `file` options: `tar -xvf T01.tar`
4. You are going to write a program that initializes an array with random numbers, then doubles the value of each of the numbers in the array, using a variation of the `doubleNum()` function that we wrote in section 1.2, program #3. Your program will use correct design techniques with modular functions.
5. You will begin by putting together a skeleton of the functions you will write for this tutorial. The provided base code already contains the `process.cc` file that contains part of the `doubleNum()` function, as well as the `checkNum()` function.

Create a new file called `main.cc` that contains two global functions: a `main()` function, as well as a `void printArray(int* arr)` function. For now, you can leave both functions with empty bodies, as our first step is to get the prototypes compiling correctly.

Create a new `init.cc` file that contains the `void initArray(int* arr)` function with an empty body.

NOTE: All the functions for this program already have their prototype defined in the `defs.h` file found in `T01.tar`. You will need to include "`defs.h`" (using the `#include` preprocessor directive) in each of the three source (`.cc`) files. Look at the content of `defs.h` to get familiar with its definitions.

6. Update the Makefile to compile and link the skeleton code. The Makefile in the base code shows how to do this with two source files. Change it so that it compiles four source files (`main.cc`, `init.cc`, `process.cc`, and `random.cc`) and links their corresponding object code.

Compile and link the code using the Makefile. Make sure that everything compiles correctly and that the `t01` executable is created.

7. Write the code for `initArray()`. This function takes an integer array as parameter, loops over every element (assuming a maximum size as defined in `defs.h`), and initializes each element to a random number between 0 and 119, using the `random()` function provided in the base code. Look at the documentation in the `random.cc` file to see how to correctly use the function.
8. Write the code for `printArray()`. This function takes an integer array as parameter (of the size indicated in `defs.h`), loops over every element, and prints its value to the screen.
9. Write the code for `main()`. This function declares an array of integers (of the size indicated in `defs.h`). It calls `initArray()` to initialize the array, then calls `printArray()` to print out its contents.

To make sure that the random number generator produces different numbers at every run, it must be seeded with a random number such as the current time. You need to `#include <cstdlib>` in your `main.cc` file, and your `main()` function must contain the following line *before* it calls the array initialization function: `srand( (unsigned)time( NULL ) );`

Compile the code and run it a few times. Check the output to make sure that the program works correctly and that different numbers are generated on every execution.

10. Update the code in the `doubleNum()` function, and change it so that it checks that the number is between 0 and 100, by calling the `checkNum()` function, *before* it doubles the number. If the number is not within those bounds, it keeps its current value and is not doubled.

Update the `main()` function so that, after the array is initialized and printed, it loops over the array and calls `doubleNum()` for every element. The array should be printed again afterwards, using the `printArray()` function.

Compile your code and run it a few times. Check the output to make sure that the program works correctly and that only numbers less than 100 are doubled.

11. Create a readme file (`README`) that includes:
 - a) a preamble (program author, purpose, list of source files)
 - b) the exact compilation command
 - c) the launching and operating instructions
12. Create a tar file called `Tutorial01.tar` that contains your source and header files, as well as your `README` file. Make sure that there are no directories, no hidden directories, and no additional files in the tar file.

Upload and submit your `Tutorial01.tar` file to the Tutorial #1 assignment in *Brightspace*.